



FIREBRAND

LYCEUM EDUCATION GROUP

Project Planning Studio

Today you plan your own project and write the first real lines of it.

What today builds towards

By the end of today you will have a plan and a working start for your own project.

1

Choose: pick a small project idea suited to your toolkit and the time available

2

Plan: write user stories, acceptance criteria and pseudocode for your idea

3

Set up: create your project files and folder structure

4

Build: write and run your first real scripts against your plan

The SDLC behind today

1

REQUIREMENTS

understand the user need, inputs are conversations and goals, output is a user story

2

DESIGN

translate the need into a plan, inputs are stories and criteria, output is pseudocode

3

BUILD

write code against the plan, inputs are pseudocode and data, output is a working script

4

TEST

check output against acceptance criteria, inputs are criteria and code, output is a pass or fail verdict

5

HANDOVER

share the result with stakeholders, inputs are the tested script and honest progress notes, output is a delivered artefact

Who does what in a software team

Business Analyst

- Gathers requirements from stakeholders
- Writes user stories and acceptance criteria
- Confirms scope before any code is written

Developer

- Translates designs and pseudocode into working code
- Follows functional and technical specifications
- Writes comments and progress notes for handover

Tester

- Checks code output against acceptance criteria
- Covers normal, boundary and invalid input cases
- Reports bugs with clear reproduction steps

Team Lead

- Owns delivery against the project plan
- Resolves blockers and prioritises work
- Reviews plans before build begins

How methodology shapes the team

Waterfall: sequential phases

- Requirements locked before design begins
- Each phase has defined inputs and outputs
- Roles are often separate, BA hands to Dev, Dev hands to Test
- Plan-first by design, change is expensive mid-project

Agile: iterative sprints

- Requirements evolve across short cycles
- Each sprint covers plan, build and test together
- Roles overlap, everyone contributes to quality
- Plan-first per sprint, change is expected and welcome

Your role in your organisation's project life-cycle

- Where does your current or target role sit in the development life-cycle at your organisation
- Which stages do you own, which do you contribute to, and which do others handle
- Does your organisation follow a named methodology, or something informal
- Think about one real project or task you have seen, which SDLC stage was it in

How far you have actually come

“

Day 1 was reading dot notation on an object. Today you are planning a project of your own.

Fourteen days of evidence.

The full toolkit

Everything you have learned sits in one of these layers.

1

FOUNDATIONS

variables, values, console.log, string concatenation

2

LOGIC

if, else if, else, comparison and logical operators

3

STRUCTURE

functions, parameters, return values

4

COLLECTIONS

arrays, objects, arrays of objects, loops

5

INTERFACES

the DOM, events, live user input and output

Console tool vs browser tool

Console-based project

- Runs in the terminal or console output only
- Good fit if you want to focus purely on logic
- Faster to build, fewer moving parts
- Examples: eligibility checkers, reports, summaries

Browser-based project

- Runs on a web page with real input and output
- Good fit if you want a visible, interactive result
- More moving parts: HTML, DOM, events
- Examples: dashboards, forms, live checkers

A finished example, read back

Here is a small, complete console-based tool built entirely from things you already know.



shift-report.js

```
1 let shiftHours = [7, 8, 6, 9, 8];
2 let totalHours = 0;
3 let longShifts = 0;
4 for (let i = 0; i < shiftHours.length; i++) {
5     totalHours = totalHours + shiftHours[i];
6     if (shiftHours[i] >= 8) {
7         longShifts = longShifts + 1;
8     }
9 }
10 ... 2 more lines in the starter file
```

Choosing your project

- 1 Pick something small enough to plan properly today.
A tiny tool that works beats a big tool that is half-planned.
- 2 Pick something that uses at least two layers of the toolkit.
A single if statement is not enough evidence, aim for logic plus data plus output.
- 3 Pick something you can explain in one sentence.
If you cannot say what it does in one sentence, it is not scoped yet.

Project ideas: checkers and validators

Ideas built mainly on conditionals, functions and validation, similar to Day 2, 4 and 11.



Shift eligibility checker

Takes hours worked and role, decides if a break is due Reuses the Day 4 eligibility pattern with new rules



Delivery slot validator

Checks a requested delivery time against opening hours Reuses Day 11's empty-input and range-guard pattern



Membership tier checker

Compares spend and visits against tier thresholds Good fit for multiple else if branches

Project ideas: reports and collections

Ideas built mainly on arrays, loops and accumulators, similar to Day 6, 7 and 9.



Weekly stock report

Loops through product objects, flags low stock, counts reorders Reuses the Day 8 and 9 warehouse and report patterns



Appointment summary

Loops through appointment objects, counts and lists no-shows Reuses the Day 9 pass count and named-list pattern



Team attendance tracker

Loops through learner records, counts at-risk attendance Direct extension of the Day 9 morning build

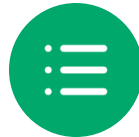
Project ideas: browser tools

Ideas built mainly on the DOM and events, similar to Day 10 and 11.



Live access checker

A form that reads a typed code and shows a clearance message Reuses the Day 11 validated access clearance checker pattern



Simple dashboard

A page showing a live count and a flagged list from an array Reuses the Day 10 delivery dashboard pattern



Booking confirmation form

A form that validates a name and slot before confirming Combines Day 11 validation with Day 10 DOM updates

Say it in one sentence

Choose your project idea and write it as a single clear sentence.

- 1 Pick one idea from the three menus, or adapt one
- 2 Write it as one sentence: what it takes in, what it decides or shows
- 3 Check the one-sentence test with a partner
- 4 Note which toolkit layers it will use



TIMEBOX

10 minutes

Individual choice, then a 2 minute partner check



OUTPUT

One written sentence describing your chosen project.

From sentence to plan

Today's plan follows the same three-step shape from Day 14.

1

USER STORY

who wants this and why

2

ACCEPTANCE CRITERIA

how you will know it works

3

PSEUDOCODE

the plain-English steps of the logic

Worked example: the shift report, planned

Here is the shift report idea taken through the full planning chain.

```
shift-report-plan.txt

1 User story:
2 As a shift supervisor, I want to see total
3 hours and long shifts, so I can check the
4 team is not over-working.
5 Acceptance criteria:
6 Given a list of shift hours, the total
7 is shown correctly.
8 A shift of 8 hours or more counts as long.
9 An empty list does not crash the tool.
```

Worked example: the pseudocode

The same shift report plan, continued: its pseudocode.

shift-report-plan.txt (continued)

```
1 Pseudocode:
2 Start totalHours at 0 and longShifts at 0
3 For each shift in the list
4     Add the shift hours to totalHours
5     If the shift is 8 or more, add 1
6     to longShifts
7 Print totalHours and longShifts
```

Write your user story

Turn your one-sentence idea into a user story using the given format.

- 1 Use the format: As a, I want, so that
- 2 Name a real role, not just you as the learner
- 3 State the goal and the reason clearly
- 4 Keep it to one or two sentences



TIMEBOX

8 minutes
Individual work



OUTPUT

**One written user story for
your chosen project.**

Define your acceptance criteria

Write three testable acceptance criteria for your project idea.

- 1 Write one criterion for the normal, expected case
- 2 Write one criterion for a boundary or edge case
- 3 Write one criterion for an invalid or empty input
- 4 Check each one can be answered pass or fail



TIMEBOX

10 minutes

**Individual work, checked
in pairs**



OUTPUT

**Three written acceptance criteria
covering normal, edge and invalid
cases.**

Write your pseudocode

Write the plain-English steps your code will follow, in order.

- 1 List each step as one plain-English line
- 2 Order the steps the way the code will actually run
- 3 Include the check from your boundary acceptance criterion
- 4 Keep it in plain English, no real syntax yet



TIMEBOX

10 minutes
Individual work



OUTPUT

A numbered list of plain-English steps describing your project's logic.

Two plans, compared

Plan A: too vague

- "Check if the delivery is valid and show a message."
- No named boundary, no explicit invalid case
- Hard to test against

Plan B: testable

- "Given a slot before 8am or after 6pm, mark as invalid."
- "Given a slot at exactly 8am, mark as valid."
- Boundary and invalid case both named clearly

Peer review your plan

Swap plans with a partner and check their acceptance criteria are testable.

- 1 Swap your user story and acceptance criteria with a partner
- 2 Check each criterion states a clear pass or fail condition
- 3 Flag any criterion that reads like Plan A, not Plan B
- 4 Suggest one improvement in plain language



TIMEBOX

10 minutes
Pairs, 5 minutes each
direction



OUTPUT

One written suggestion given
and received on acceptance
criteria.

The planning document, assembled

Your morning's output is one document holding all three artefacts together.

Project name

one clear title for your idea

User story

the role, goal and reason

Acceptance criteria

three testable pass or fail statements

Pseudocode

the ordered plain-English steps

Build your planning document

Combine your project name, user story, acceptance criteria and pseudocode into one document.

- 1 Create a new plain text or document file
- 2 Add your project name as a heading
- 3 Paste in your user story, acceptance criteria and pseudocode
- 4 Read it once, start to finish, as if you are handing it over



TIMEBOX

15 minutes
Individual work



OUTPUT

One complete planning
document ready to code from
this afternoon.

What the plan actually protects you from

“

A plan does not slow down building. Skipping one does.

The lesson behind every debugging session this programme has covered.

Morning recap



my plan includes a user story naming a real role

The role, goal and reason are all stated in one or two sentences



my acceptance criteria include a boundary case

The exact edge value, not just a general description, is named



my pseudocode may not be in build order yet

Read each step aloud in sequence and check it flows logically



my project scope might be too large for one day

If there are more than five pseudocode steps, consider cutting to the core feature



Lunch

Back at the time given by your trainer, ready to build.



FIREBRAND

LYCEUM EDUCATION GROUP

Afternoon: from plan to running code

This afternoon you set up your project and write its first real scripts.

Working in a team: what good looks like

Even solo builds reflect team habits. These are the practices that make a team work.



Shared understanding

Everyone reads the same planning document before writing a single line Prevents duplicated work and contradictory assumptions



Small, testable steps

Each person works on one piece at a time and tests before moving on Makes it easy to spot who introduced a bug and when



Honest progress notes

Comments and notes tell teammates what is done and what remains Handover works because nobody has to guess



Respectful review

Feedback focuses on the plan and the code, not the person A better suggestion improves the whole product, not just one file

Reread your own pseudocode

Reread your pseudocode from this morning and number each step.

- 1 Open your planning document
- 2 Number each pseudocode line in build order
- 3 Circle the first step you will code first
- 4 Note which toolkit pattern that first step needs



TIMEBOX

6 minutes
Individual work



OUTPUT

**A numbered pseudocode list
with your first coding step
identified.**

Setting up your project

Before writing logic, get your files in place using this simple structure.

1

CREATE A FOLDER

name it clearly after your project

2

CREATE YOUR MAIN SCRIPT FILE

name it something.js

3

ADD STARTING DATA

the values or objects your logic will work on

4

ADD A PLAN COMMENT

paste your pseudocode in as comments at the top

What a scaffolded starter file looks like

Here is the shift report file before any real logic is written, just the plan as comments.

```
shift-report-starter.js

1 // Start totalHours at 0 and longShifts at 0
2 // For each shift in the list
3 //     Add the shift hours to totalHours
4 //     If the shift is 8+, add 1 to longShifts
5 // Print totalHours and longShifts
6 let shiftHours = [7, 8, 6, 9, 8];
```

Set up your own project file

Create your project folder, script file, starting data and comment scaffold.

- 1 Create a folder named after your project
- 2 Create a new .js file inside it
- 3 Paste your numbered pseudocode in as comments at the top
- 4 Add one line of starting data below the comments



TIMEBOX

12 minutes
Individual work



OUTPUT

**A new script file with a
pseudocode comment scaffold
and starting data.**

Turning the first line into code

The first pseudocode comment becomes the first real line underneath it.

```
shift-report-starter.js

1 // Start totalHours at 0 and longShifts at 0
2 let totalHours = 0;
3 let longShifts = 0;
4 // For each shift in the list
5 //   (loop and conditional come next)
6 let shiftHours = [7, 8, 6, 9, 8];
```

Code your first pseudocode step

Write the real code for the very first line of your pseudocode.

- 1 Find the first pseudocode comment in your file
- 2 Write one or two lines of real code directly beneath it
- 3 Run the file and check nothing errors
- 4 Leave the comment in place above your new code



TIMEBOX

10 minutes
Individual work



OUTPUT

**Your first working line or two of
real code beneath its
pseudocode comment.**

The next step: adding the loop

JavaScript

```
1 for (let i = 0; i < shiftHours.length ...  
2   totalHours += shiftHours[i];  
3   if (shiftHours[i] >= 8) {  
4     longShifts += 1;  
5   }  
6 }
```

1

Loop label

for each shift in the list

2

Accumulator

add the shift hours as we go

3

Conditional counter

flag it if it meets the condition

Code your next pseudocode step

Write the real code for your next pseudocode line, whatever it needs.

- 1 Find the next unwritten pseudocode comment in your file
- 2 Write the code it describes beneath it
- 3 Run the file after each small change
- 4 Ask your trainer if the pattern feels unfamiliar



TIMEBOX

15 minutes

**Individual work, trainer
circulating**



OUTPUT

**At least one more working
section of code matching your
pseudocode.**

The only two questions that matter right now

“

Does it run, and does it match what your pseudocode said it should do.

The whole of software testing, at its simplest.

A common early bug, spotted

Two versions of the same loop, one with a common off-by-one slip

shift-report-buggy.js

```
1 let totalHours = 0;
2 for (let i = 0; i <= shiftHours.length; i++)
3   totalHours += shiftHours[i];
4 }
```

shift-report-fixed.js

```
1 let totalHours = 0;
2 for (let i = 0; i < shiftHours.length; i++)
3   totalHours += shiftHours[i];
4 }
```

Check your own loop boundary

If your code includes a loop, check its condition against this common slip.

- 1 Find any for loop in your file so far
- 2 Check the condition uses less than, not less than or equal to
- 3 Run the file and read the output carefully
- 4 Fix and rerun if you spot an extra or missing item



TIMEBOX

6 minutes
Individual work



OUTPUT

A confirmed correct loop boundary, or a fix applied and rerun.

Testing against your own criteria

From here, build in this same small loop until your pseudocode is complete.

- 1 Write the next small piece of code
- 2 Run the file
- 3 Check the output against your acceptance criteria
- 4 Fix or adjust if it does not match
- 5 Move to the next pseudocode step

The main build: your project so far

Continue building your project, working through your pseudocode step by step.

- 1 Keep working down your numbered pseudocode comments
- 2 Test after every small addition, not just at the end
- 3 Check output against your acceptance criteria as you go
- 4 Flag your trainer if you are stuck for more than a few minutes



TIMEBOX

20 minutes

**Individual work, trainer
circulating actively**



OUTPUT

**A partially or fully working script
matching at least some of your
pseudocode.**

Policies, data and your code

Real projects carry obligations that go beyond syntax.

1

GDPR and personal data: if your project handles names, IDs or records, how it stores and outputs that data is governed by law

2

Organisational policies: most employers have rules about how code is written, stored, reviewed and deployed, check yours

3

Security by habit: avoid hardcoding sensitive values, keep credentials out of scripts, and never log personal data to the console

4

Secure working environment: lock screens when away, do not commit work to shared drives without knowing who has access

5

Compliance is not optional: a working script that leaks personal data is not a finished script, it is a liability

Check your project against data obligations

Read your own project plan and code through a data and compliance lens.

- 1 Identify any variable that holds a name, ID or personal detail
- 2 Check nothing sensitive is printed directly to the console
- 3 Note one policy at your employer this project would need to follow



TIMEBOX

8 minutes
Individual work



OUTPUT

A short written note on data obligations relevant to your project.

Version control and deployment basics

Saving a file is not the same as managing code safely.

1

Version control: tools like Git track every change you make, so you can see what changed and when, and undo mistakes safely

2

Commit messages: a short, honest note on each save tells your team what you did and why, not just that you saved

3

Source control: a shared repository like GitHub or Azure DevOps is where your team's code actually lives, not just your laptop

4

Deployment: moving tested code from your local machine into a staging or production environment is a controlled, documented step

5

Your organisation's approach: most employers have a defined process for all of these, following it is part of your role, not optional

Continue your build

Keep working through your remaining pseudocode steps.

- 1 Pick up from where you paused in your pseudocode
- 2 Write and test one more section of code
- 3 Check it against your acceptance criteria
- 4 Add a comment noting anything left unfinished



TIMEBOX

15 minutes
Individual work



OUTPUT

**Additional working code,
plus honest notes on what
remains.**

Marking honest progress in comments

A short, honest comment noting what is not yet built is genuinely useful, not a weakness.

```
shift-report-progress.js

1 let totalHours = 0;
2 let longShifts = 0;
3 for (let i = 0; i < shiftHours.length; i++) {
4     totalHours += shiftHours[i];
5     if (shiftHours[i] >= 8) {
6         longShifts += 1;
7     }
8 ... 3 more lines in the starter file
```

Add your own progress comment

Add one honest comment at the bottom of your file noting what remains.

- 1 Scroll to the bottom of your current file
- 2 Add a comment starting with Not yet built:
- 3 List what is still missing in plain English
- 4 Save the file



TIMEBOX

5 minutes
Individual work



OUTPUT

**A saved file with an honest
note on remaining work.**

Check your project against the toolkit



my file runs without an error right now

My last test run produced output with no red error text



my code follows my pseudocode order

Each section beneath a comment does what that comment describes



my acceptance criteria are not all tested yet

Note which of the three from this morning are still unchecked



my edge case is not yet handled

The boundary or empty-input case from this morning may still be open

Close one gap

Pick one flagged item from your checklist and try to close it.

- 1 Choose your most important unchecked or flagged item
- 2 Write the code needed to address it
- 3 Test it against the relevant acceptance criterion
- 4 Update your progress comment if it is now resolved



TIMEBOX

10 minutes
Individual work



OUTPUT

**One previously flagged item
addressed, tested, or clearly still
noted.**

What today's plan prevented

A clear one-sentence project scope from the start

Testable acceptance criteria written before any code

Pseudocode mapping directly onto build order

Comments marking honest progress throughout

Outcome: a working, testable start with no wasted rebuilding

Plan-first vs code-first

Code-first, no plan

- Starts typing immediately
- Discovers requirements while debugging
- Often rebuilds the same section twice

Plan-first, today's approach

- Writes user story and criteria before code
- Tests against a known standard as you go
- Builds each section once, deliberately

Continuing to grow after this programme

Today is a start, not a finish line. Professional development continues from here.

1

Reflect regularly: set aside time each month to review what skills you have used and what gaps you have noticed in real work

2

Seek feedback: ask a team lead or mentor to review one piece of your code and explain their suggestions

3

Read widely: blogs, documentation and open source code are how most developers keep pace with a fast-moving field

4

Contribute back: sharing a working pattern, a comment, or a review with a colleague builds the team's knowledge as well as your own

5

Log your learning: apprenticeship portfolios are one form of this habit, but keeping a simple log of what you tried and what you learned is useful long after the programme ends

In your own words

Write two or three sentences explaining your project to a non-technical friend.

- 1 Write what your project does in plain English
- 2 Avoid naming any code terms like loop or array
- 3 Read it aloud to a partner



TIMEBOX

8 minutes

Individual write, then pair share



OUTPUT

A short plain-English explanation of your project.

Exit ticket

Answer four short questions about today before you finish.

- 1 Name one thing your project does correctly right now
- 2 Name one toolkit skill from an earlier day you reused today
- 3 Name one thing you would do differently next session
- 4 Rate your confidence continuing this build, one to five



TIMEBOX

8 minutes

Individual written work



OUTPUT

**Four written answers
reflecting on today's plan
and build.**

Day complete

You planned a real project today, and you started building it for real.



You reused fourteen days of skills in one coherent plan



You wrote testable acceptance criteria before touching code



You built in small, tested steps rather than guessing



You have a working start and an honest, clear next step